Open Source Astrodynamic Mission Planning

Development Roadmap

Phases 1–5 + Stretch Goals

Precision Newtonian • CR3BP & Low-Energy Transfers • Star Field & Astro-Navigation • GR (stretch)

AGPL-3.0 — HortDog — May 14, 2026

Contents

1 Executive Summary **3**

2 Phase 2 — Precision Newtonian & Mission Design **4**
- 2.1 Goals . . . 4
- 2.2 2a — Dynamics Fidelity . . . 4
  - 2.2.1 Higher-Order Gravity . . . 4
  - 2.2.2 Atmospheric Drag . . . 4
  - 2.2.3 Solar Radiation Pressure . . . 4
  - 2.2.4 Third-Body Gravity . . . 4
  - 2.2.5 Symplectic Integrator . . . 5
  - 2.2.6 New API Flags . . . 5
  - 2.2.7 Tests . . . 5
- 2.3 2b — TLE Ingest . . . 5
- 2.4 2c — Δv Manoeuvre Engine . . . 5
  - 2.4.1 Data Model . . . 5
  - 2.4.2 Analytical Solvers . . . 6
  - 2.4.3 Numerical Optimiser . . . 6
- 2.5 2d — Real-time WebSocket Streaming . . . 6
- 2.6 2e — Frontend . . . 6
- 2.7 Phase 2 Sequencing . . . 7

3 Phase 3 — Sandbox Integration & Lunar System **8**
- 3.1 Goals . . . 8
- 3.2 3.1 — Perturbation editor expansion (shipped) . . . 8
- 3.3 3.2 — Solver panel (shipped) . . . 8
- 3.4 3.3 — Central-body selector + lunar constants . . . 8
- 3.5 3.4 — Moon rendering . . . 8
- 3.6 3.5 — Cislunar TLI demo . . . 9
- 3.7 3.6 — Polish . . . 9
- 3.8 3.x — Known limitations . . . 9
- 3.9 3 — Status & tests . . . 9

4 Phase 4 — Rotating Frames, CR3BP & Low-Energy Transfers **10**
- 4.1 Goals . . . 10
- 4.2 Pillar A — Frame infrastructure . . . 10
  - 4.2.1 4.1 Frame definitions . . . 10
  - 4.2.2 4.2 Frame-aware propagator and visualization . . . 10
- 4.3 Pillar B — CR3BP dynamics . . . 10
  - 4.3.1 4.3 CR3BP propagator . . . 10
  - 4.3.2 4.4 Lagrange points + Jacobi constant . . . 11
  - 4.3.3 4.5 Periodic orbit families (hardest single item) . . . 11
  - 4.3.4 4.6 Invariant manifolds . . . 11
- 4.4 Pillar C — Low-energy transfers . . . 11
  - 4.4.1 4.7 Manifold-guided transfers . . . 11
  - 4.4.2 4.8 Ballistic-capture diagnostic . . . 11
- 4.5 Pillar C visualization . . . 11
  - 4.5.1 4.9 Frontend integration . . . 11
- 4.6 Sequencing . . . 12

4.7 Hard tradeoffs decided up front . . . 12

5 Phase 5 — Real Star Field & Astro-Navigation 13
5.1 Goals . . . 13
5.2 Pillar A — Star catalogue & renderer . . . 13
5.2.1 5.1 Catalogue ingest . . . 13
5.2.2 5.2 WebGPU star renderer . . . 13
5.2.3 5.3 Bright movers . . . 13
5.3 Pillar B — Astro-navigation algorithms . . . 14
5.3.1 5.4 Star-tracker simulation . . . 14
5.3.2 5.5 Lost-in-space attitude solver . . . 14
5.3.3 5.6 Optical landmark / planet-limb navigation . . . 14
5.3.4 5.7 Closed-loop "navigate by stars" demo . . . 14
5.4 Pillar C — Frontend integration . . . 15
5.4.1 5.8 Sky toggle + sandbox . . . 15
5.5 Sequencing . . . 15
5.6 Hard tradeoffs decided up front . . . 15

6 Future / Stretch Goals 16
6.1 Stretch A — General Relativity . . . 16
6.1.1 A.1 Post-Newtonian corrections (bridge) . . . 16
6.1.2 A.2 Full GR geodesic integrator . . . 16
6.1.3 A.3 Tetrad thrust model . . . 16
6.1.4 A.4 Metric pre-bake cache (Ray) . . . 16
6.1.5 A.5 Frontend . . . 16
6.2 Stretch B — Tesseral gravity . . . 17
6.3 Stretch C — Ground-track & visibility analysis . . . 17
6.4 Stretch D — Conjunction analysis . . . 17
6.5 Stretch E — Multi-spacecraft & constellation propagation . . . 17

7 Let Your Creativity Go Wild 18
7.1 Cinematic & storytelling modes . . . 18
7.2 Live data & real-world coupling . . . 18
7.3 Wild physics / alternate sandboxes . . . 18
7.4 New audiences & interfaces . . . 18
7.5 Asteroid & deep-space . . . 19
7.6 In-browser optimisation playground . . . 19

8 Overall Timeline 20
8.1 Phase comparison . . . 20
8.2 Critical path . . . 20

9 Conventions Reference 21

Executive Summary

Phase 1 delivered the project skeleton: a Newtonian + J2 propagator with DOP853 integration, an open-loop launch simulator, a SPICE kernel wrapper with reproducibility-locked manifests, and a WebGPU 3D viewer showing three orbital scenarios.

Phase 2 (*shipped*) keeps the physics Newtonian but raises the fidelity to the level needed for real mission design: higher-order gravity (J2–J6), atmospheric drag (exponential + MSIS), solar radiation pressure, third-body perturbations, symplectic integrators (Verlet, Yoshida-4), finite-burn modelling, TLE ingest, a Lambert / CasADi trajectory optimizer, NDJSON streaming, and a full manoeuvre-editor frontend.

Phase 3 (*shipped*) brings every backend capability to the sandbox frontend (J2–J6, drag with MSIS, SRP, third-body Moon/Sun, finite burns, four solver panels covering Hohmann/Lambert/multi-burn NLP/TLE), adds a central-body selector with full Moon support (canonical μ, R, J_n, ω), wires SPICE ephemerides into the renderer as a secondary-body wireframe, and ties it together with an end-to-end cislunar TLI demo.

Phase 4 (*shipped*) extends the dynamics layer with the circular-restricted three-body problem (CR3BP), Lagrange-point computation, planar Lyapunov periodic orbits via differential correction, invariant manifold tubes, EM-synodic rotating-frame visualization, and the weak-stability-boundary (WSB) capture diagnostic — the foundations for low-energy transfers and ballistic capture into lunar orbit.

Mission-planner polish (*shipped*) wraps the editor: preset-orbit library (ISS/LEO/SSO/GTO/GEO/Molniya/lu seeds), orbital elements editor, integrator + epoch + SPICE-status controls, live $\Sigma|\Delta v|$ budget, save / load mission JSON, autosave to `localStorage`, CSV trajectory export, multi-burn inertial→RIC auto-conversion, event-timeline strip, in-editor Lyapunov / WSB / launch sub-panels, animated time scrub with solid past / dashed future, real Natural Earth coastline overlay.

Phase 5 (*next*) renders a real **star field** from the Hipparcos / Yale Bright Star catalogue and adds **astro-navigation**: star-tracker simulation, lost-in-space attitude solvers, optical landmark navigation, and a closed-loop "navigate by stars" demo.

Future / Stretch Goals keep the door open for ambitious follow-ups: post-Newtonian and full General Relativity propagation (BCRS geodesics, tetrad thrust, Ray-backed metric pre-bake cache), tesseral gravity, ground tracks / visibility analysis, conjunction analysis against TLE catalogues.

"Let your creativity go wild" closes the document with a deliberately open-ended list of blue-sky directions — tour mode, live ISS tracking, generative trajectory design, asteroid mining, solar sails, WebXR, audio synthesis, alternate-physics sandboxes — where the only constraint is "it should be cool."

Convention lock: UTC ISO-8601 on the wire; TDB seconds since J2000 (= SPICE ET) internally. All SI units in the engine. **Licence: AGPL-3.0** (switched from Apache-2.0 in Phase 4 polish; network-service deployments inherit source-disclosure obligations). SPICE kernels and the Natural Earth coastline are now tracked in-tree so a fresh clone runs with zero network fetches.

Phase 2 — Precision Newtonian & Mission Design

Goals

A mission designer should be able to: load a TLE or specify an initial state, apply atmospheric drag and solar radiation pressure, add impulsive Δv burns, solve a Lambert transfer or run a fuel-optimal CasADi optimisation, and watch the result stream live in the 3D viewer.

2a — Dynamics Fidelity

Higher-Order Gravity

Extend `oamp.dynamics.newtonian` with zonal harmonics J_3–J_6 (Vallado §8.7). The acceleration from J_n is:

$$\mathbf{a}_{J_n} = -\frac{\mu}{r^2}\frac{n+1}{2}\left(\frac{R_\oplus}{r}\right)^n J_n \nabla P_n(\sin\phi) \tag{1}$$

where P_n is the Legendre polynomial and ϕ the geocentric latitude. Tesseral (C_{nm}, S_{nm}) terms are planned but deferred to Phase 2b+.

Atmospheric Drag

Replace the exponential density model in `launch.py` with NRLMSISE-00 via the `nrlmsise00` Python package. The drag acceleration is:

$$\mathbf{a}_{\text{drag}} = -\tfrac{1}{2}\,\rho(h)\,C_D\,\frac{A}{m}\,|\mathbf{v}_{\text{rel}}|\,\mathbf{v}_{\text{rel}} \tag{2}$$

where $\mathbf{v}_{\text{rel}}$ accounts for Earth's rotation ($\boldsymbol{\omega}_\oplus \times \mathbf{r}$). Vehicle cross-section A and C_D are added to the propagation request body.

Solar Radiation Pressure

Cannonball model with cylindrical shadow cones for Earth and Moon:

$$\mathbf{a}_{\text{SRP}} = \nu\frac{P_\odot}{c}\frac{A_{\text{eff}}}{m}\hat{\mathbf{r}}_\odot \tag{3}$$

where $\nu \in [0,1]$ is the shadow function (computed via SPICE body positions already available) and $P_\odot = 4.56 \times 10^{-6}$ N m^{-2} at 1 AU.

Third-Body Gravity

Moon and Sun perturbations using SPICE `body_state()` (already implemented) with point-mass attractions and the indirect term:

$$\mathbf{a}_{3\text{b}} = \sum_k \mu_k\left(\frac{\mathbf{r}_k - \mathbf{r}}{|\mathbf{r}_k - \mathbf{r}|^3} - \frac{\mathbf{r}_k}{|\mathbf{r}_k|^3}\right) \tag{4}$$

Symplectic Integrator

Add SABA2 (symplectic Birkhoff–Adams) alongside DOP853 for long-arc propagation where energy conservation over months matters more than step-size adaptivity. Selectable via `integrator: "dop853" | "saba2"` in the request.

New API Flags

Field	Type	Effect
`drag`	`bool`	NRLMSISE-00 drag
`srp`	`bool`	Solar radiation pressure
`third_body`	`list[str]`	Bodies e.g. `["moon","sun"]`
`jn_max`	`int`	Max zonal harmonic (2–6)
`integrator`	`str`	`"dop853"` or `"saba2"`
`vehicle`	`VehicleModel`	m, A, C_D, C_R

Tests

- Energy drift $< 10^{-9}$ relative over 30-day arc (SABA2)
- Nodal regression matches NRLMSISE drag prediction within 5%
- SRP produces correct signature in inclination evolution
- Moon third-body matches SPICE-derived perturbation to < 1 m/day

2b — TLE Ingest

Add `GET /tle?norad={id}` which:

1. Fetches the current TLE from Celestrak
2. Parses with `sgp4` (already evaluable from `conda-forge`)
3. Propagates to epoch via SGP4
4. Converts mean-element state to osculating Cartesian (Brouwer transform)
5. Returns an `InitialState` compatible with `POST /propagate`

A manifest of pinned test TLEs (NORAD ID + epoch + expected state checksum) mirrors the SPICE kernel manifest pattern for reproducible CI.

2c — Δv Manoeuvre Engine

Data Model

```
class Maneuver(BaseModel):
    t_tdb: float                # TDB seconds since J2000
    dv_ric: list[float]         # [radial, in-track, cross-track], m/s
```

`POST /propagate` accepts an optional `maneuvers: list[Maneuver]`; the integrator applies instantaneous velocity kicks at each epoch by transforming RIC → ECI and restarting the ODE.

Analytical Solvers

Endpoint	Description
`POST /optimize/hohmann`	Closed-form two-impulse Hohmann transfer. Returns Δv_1, Δv_2, transfer time.
`POST /optimize/lambert`	Universal-variable Lambert solver (Izzo/Gooding algorithm) for arbitrary two-point boundary-value transfer.

Numerical Optimiser

`POST /optimize/trajectory` uses CasADi (already in the `default` pixi environment) to solve the fuel-optimal NLP:

$$\min_{\{\Delta \mathbf{v}_k\}} \sum_k |\Delta \mathbf{v}_k| \quad \text{s.t.} \quad \mathbf{x}_{k+1} = \phi(\mathbf{x}_k + \Delta \mathbf{v}_k,\, \Delta t_k),\ \mathbf{x}_N = \mathbf{x}_{\text{target}} \tag{5}$$

The propagation ϕ is the DOP853 integrator wrapped as a CasADi callback. Warm-starting from the Lambert solution typically converges in < 30 iterations.

2d — Real-time WebSocket Streaming

Replace the echo-only `/ws` channel with a streaming propagator. The protocol uses msgpack (already a dependency):

1. Client sends {`scenario_id, t_start, t_end, dt, config`}
2. Server propagates in chunks of N steps on a Ray actor (or asyncio for small jobs)
3. Each chunk is msgpack-encoded and sent as a binary WebSocket frame
4. A terminal {`done: true, elapsed_ms`} frame closes the stream

This enables the frontend time-scrubber to show live progress for long multi-day propagations.

2e — Frontend

Feature	Detail
Manoeuvre editor	Click on trajectory to insert a burn node; drag the arrow glyph to adjust Δv magnitude and direction in the RIC frame.
Time scrubber	Slider replays the trajectory; a spacecraft marker moves along the path. Shows t_{UTC}, r, v, and remaining Δv budget.
Multi-trajectory overlay	Compare baseline vs optimised trajectories side-by-side with per-trajectory colour coding and a difference-norm plot in a side panel.
Velocity / thrust vectors	Short arrow glyphs at the spacecraft position; length proportional to $\lvert\mathbf{v}\rvert$ or $\lvert\mathbf{f}\rvert$ with configurable scale.
WebGL2 fallback	2D ground-track canvas (equirectangular) for browsers without WebGPU. Uses the same trajectory data from the REST API.
Scenario save/load	JSON export of full mission state (initial conditions, manoeuvres, vehicle model, propagation config). Drag-and-drop import.

Phase 2 Sequencing

Work item	Est. duration	Dependency
2a Drag + third-body	2–3 weeks	Phase 1
2b TLE ingest	1 week	Phase 1
2c Lambert + CasADi NLP	2–3 weeks	2a
2d WebSocket streaming	1 week	Phase 1
2e Frontend	3–4 weeks	2c, 2d (parallel)
Total	**9–12 weeks**	

Phase 3 — Sandbox Integration & Lunar System

Goals

Make every backend capability visible and editable in the WebGPU sandbox, extend the central-body abstraction to the Moon, and use SPICE ephemerides to render the Moon at its inertial position — producing an end-to-end cislunar transfer demo that exercises the full Phase 2 stack (higher-order gravity, third-body perturbation, finite burns, Lambert).

3.1 — Perturbation editor expansion (shipped)

The maneuver editor sidebar gained a full Perturbations section exposing every flag the backend already accepts:

- Zonal harmonics dropdown ($J_2 \dots J_6$) bound to `PropagateRequest.jn_max`
- Atmospheric drag with model selector (`exponential` — `msis`); vehicle inputs share a single section
- Solar radiation pressure (cannonball) with C_r, area
- Third-body checkboxes for Moon and Sun (forwarded as `third_body: ["MOON", "SUN"]`)
- Live "active forces" readout displaying what the server actually applied (`response.perturbations[]`)

3.2 — Solver panel (shipped)

A single sidebar section with a solver-type dropdown and a dynamically rendered form:

Solver	Inputs	Loads into editor
Hohmann	r_1, r_2, μ	2 impulsive maneuvers, IC at LEO
Lambert	$\vec{r}_1$, $\vec{r}_2$, TOF	IC $= (\vec{r}_1, \vec{v}_1^{\text{Lambert}})$, duration = TOF
Multi-burn NLP	N epochs, target $\vec{r}_f, \vec{v}_f$	v list + convergence stats (inertial; displayed)
TLE	NORAD ID *or* line1/line2 (+ `at_utc`)	ECI state at epoch, period auto-set

Each solver writes results back to `editorState` and auto-propagates, so the user sees the orbit immediately.

3.3 — Central-body selector + lunar constants

`PropagateRequest.body_name: "EARTH" | "MOON" | "SUN"` resolves to the canonical `Body` constant (correct μ, R, J_n, ω); the legacy `mu`/`body_radius` fields are kept for back-compat. Lunar zonal harmonics $J_2 \dots J_6$ are populated from the GRGM1200A low-order solution. The frontend dropdown repopulates the initial state with a sensible low circular orbit at the body's default altitude and clears drag/SRP (which require an atmosphere).

3.4 — Moon rendering

The WebGPU renderer gained `setSecondaryBody(position, radius, color)`: a low-resolution wireframe sphere translated to the inertial Moon position. A new endpoint `POST /spice/ephemeris` accepts a list of TDB-second offsets from a UTC epoch and returns the corresponding body states — batching what `/spice/state` did for a single time. The frontend samples Moon positions at

$\sim$60 points across the active trajectory and advances the Moon as the time scrubber moves (binary-search lookup).

3.5 — Cislunar TLI demo

A top-bar button that ties the whole stack together:

1. Query Moon position at the demo epoch from SPICE
2. Solve Lambert: 200 km LEO $\rightarrow$ Moon position in 3.5 d
3. Load TLI v as an impulsive RIC maneuver
4. Propagate with J_2 + third-body Moon
5. Render trajectory + Moon wireframe + apse markers

Graceful degradation: if SPICE kernels aren't loaded the status bar prompts the user to run `pixi run kernels` rather than failing silently.

3.6 — Polish

- **Camera**: adaptive near/far ($0.005 \cdot d \ldots 500 \cdot d$) keeps `depth24plus` precision usable across the whole zoom range; wheel-zoom band expanded to $10^{-3} \ldots 10^{4}$ scene units
- **Camera target switching**: orbit-camera target switchable between central body / spacecraft / Moon via a footer dropdown
- **Apse markers**: tiny magenta/yellow crosses at periapsis / apoapsis. Detection via $\dot{r}$ zero-crossings (not triplet extremum) and suppressed when $(r_{\max} - r_{\min})/\bar{r} < 10^{-3}$ to avoid spurious markers on near-circular orbits
- **Altitude chart**: $|\vec{r}|(t)$ polyline in the footer with a vertical cursor that tracks the time scrubber

3.x — Known limitations

- Multi-burn NLP returns inertial Δv; the editor speaks RIC, so it displays results rather than round-tripping. A future `dv_inertial` field on the Maneuver model would close this.
- Moon ephemeris is sampled only at the configured demo epoch (`2026-01-01T00:00:00`); a per-trajectory configurable t_0 is a small follow-up.
- Frame is always inertial J2000. Phase 4 introduces the synodic-frame option needed for periodic-orbit analysis.

3 — Status & tests

All 43 backend tests pass; ruff clean; frontend type-check clean.

Phase 4 — Rotating Frames, CR3BP & Low-Energy Transfers

Goals

Add the dynamics, frames, and visualization needed to design and visualize *low-energy* (manifold-guided) transfers and *ballistic capture* trajectories. The Newtonian path remains the fast default; CR3BP is opt-in via a frame selector and a separate propagator family.

The deliverable is: "the user can pick an Earth–Moon halo orbit by energy, see the unstable manifold tube reaching Earth, and choose an initial state on that tube that ballistically captures into a low lunar orbit."

Pillar A — Frame infrastructure

4.1 Frame definitions

A `Frame` dataclass in `backend/oamp/frames.py` (new file) with time-dependent rotation + translation:

- **J2000**: inertial (current default)
- **EM_SYNODIC**: rotating with the Earth–Moon line, origin at the system barycenter — the Moon and Earth are stationary
- **SE_SYNODIC**: rotating with the Sun–Earth line; required for Weak-Stability-Boundary transfers
- **BCI_EARTH** / **BCI_MOON**: body-centered J2000
- **BCR_***: body-centered rotating, surface-relative

Each conversion is a 3×3 rotation $R(t)$ plus an origin translation $\Delta\vec{r}(t)$, plus the velocity correction $\vec{v}' = R(\vec{v} - \boldsymbol{\omega} \times \vec{r}) - \dot{\Delta\vec{r}}$.

4.2 Frame-aware propagator and visualization

- `PropagateRequest.output_frame` — backend transforms the result before returning
- `POST /transform/states` — standalone batch transform endpoint
- Frontend frame selector dropdown coupled to `output_frame` and to the renderer's "what is at origin"

Pillar B — CR3BP dynamics

4.3 CR3BP propagator

Non-dimensional equations of motion in the synodic frame with mass ratio $\mu = m_2/(m_1 + m_2)$. In file `backend/oamp/dynamics/cr3bp.py`. JAX-friendly RHS so we can JIT and `vmap` for batch trajectory studies (this links to the GPU acceleration work):

$$\begin{aligned}
\ddot{x} &= \quad 2\dot{y} + x - (1-\mu)\frac{x+\mu}{r_1^3} - \mu\frac{x-1+\mu}{r_2^3} \\
\ddot{y} &= -2\dot{x} + y - (1-\mu)\frac{y}{r_1^3} - \mu\frac{y}{r_2^3} \\
\ddot{z} &= -(1-\mu)\frac{z}{r_1^3} - \mu\frac{z}{r_2^3}
\end{aligned} \tag{6}$$

4.4 Lagrange points + Jacobi constant

- Newton solve for collinear L_1, L_2, L_3 from the quintic; exact for triangular L_4, L_5
- `jacobi_constant(state, μ)` as a pseudo-energy invariant; conserved to $\sim 10^{-10}$ over a closed periodic orbit, useful as an integration-error gauge
- Endpoint `GET /cr3bp/lagrange?mu=...`

4.5 Periodic orbit families (hardest single item)

- Planar Lyapunov around L_1, L_2 — linearised initial guess + differential correction (Newton on the period-crossing residual)
- 3D halo orbits — Richardson 3rd-order analytical seed, then differential correction
- Vertical Lyapunov (out-of-plane)
- **Family continuation**: pseudo-arclength on energy/period to walk along a family
- Endpoint `POST /cr3bp/periodic-orbit {type, L, energy}`

4.6 Invariant manifolds

- Monodromy matrix $\Phi(T)$ around a periodic orbit
- Eigendecomposition $\rightarrow$ stable ($|\lambda| < 1$) and unstable ($|\lambda| > 1$) directions
- Integrate the stable manifold backward and the unstable manifold forward from a sampling of points along the orbit ($\sim$100 trajectories typical)
- Endpoint `POST /cr3bp/manifold {orbit, direction, branch, duration}`

Pillar C — Low-energy transfers

4.7 Manifold-guided transfers

- **Earth–Moon L_1/L_2 transit**: depart LEO $\rightarrow$ unstable manifold of L_1/L_2 halo $\rightarrow$ arrive at the lunar SOI without a capture burn
- **WSB (Belbruno) transfer**: depart Earth $\rightarrow$ Sun–Earth manifold tube $\rightarrow$ swing through Sun–Earth L_1/L_2 $\rightarrow$ arrive at Earth–Moon L_2 manifold $\rightarrow$ ballistic capture into lunar orbit. $\sim$90–110 d TOF, $\sim$10–20% Δv savings vs. Hohmann (Hiten, GRAIL, LADEE precedent).
- Endpoint `POST /transfer/low-energy {depart, target, type}`

4.8 Ballistic-capture diagnostic

Sample many initial conditions near the lunar L_1/L_2 manifolds, propagate backward in the BCR4BP (Sun perturbation included), classify each as "captured" (lunar bound for ≥ 3 months) or "escapes". The boundary between these classes is the **Weak Stability Boundary**. Visualize as a colour-coded map in the synodic frame — the iconic Belbruno plot.

Pillar C visualization

4.9 Frontend integration

- Frame selector coupled to `output_frame` on every propagate call; renderer "central body" becomes the barycenter when synodic
- $L_1 \ldots L_5$ markers (small $\times$ glyphs) drawn at their synodic-frame positions

- Periodic-orbit catalogue browser — preset halo/Lyapunov entries by energy, click to load
- Manifold tube rendering: many semi-transparent trajectories, colour-coded by stable/unstable and ± branch
- Live Jacobi-constant readout in the footer (sanity check; drift diagnoses integration error or 4-body perturbation)

Sequencing

4.1 & 4.3	foundational, independent, ∼2–3 d each
4.2 (frame propagator)	needs 4.1; ∼1 d
4.4 (Lagrange + Jacobi)	needs 4.3; ∼1 d
4.5 (periodic orbits)	needs 4.4; ∼5–7 d (DC robustness)
4.6 (manifolds)	needs 4.5; ∼3–4 d
4.7 (transfers)	needs 4.6 + 4.2; ∼3 d
4.8 (WSB diagnostic)	needs 4.6 + BCR4BP RHS; ∼4–5 d
4.9 (visualization)	threads through all of the above; ∼3–4 d
Total	**∼5–6 weeks**

Hard tradeoffs decided up front

- **CR3BP vs full ephemeris**: do all family computation in pure CR3BP; refine into the SPICE-ephemeris model as a forward- shooting follow-up. Pure CR3BP first.
- **JAX vs SciPy for batch manifolds**: build on SciPy DOP853 first to match the existing code path; expose a JAX `lax.scan` + `vmap` fast path under the `gpu` pixi feature in a follow-up.
- **Synodic-frame UI complexity**: do the inertial→synodic transform on the backend (`output_frame` in the request); renderer stays a dumb pipeline.
- **Discoverability**: add a small glossary section in the sidebar covering Jacobi, halo, manifold, WSB — these terms are unusable to anyone outside the field.
- **BCR4BP scope**: only used by 4.8 (ballistic capture diagnostic). Don't try to do periodic orbits or generic propagation in BCR4BP — CR3BP is enough everywhere else.

Phase 5 — Real Star Field & Astro-Navigation

Goals

Two intertwined deliverables:

1. **Star field**: render a real celestial sphere from a standard astrometric catalogue so the background of the WebGPU scene shows the actual sky — the same view a spacecraft camera would see.
2. **Astro-navigation**: build the algorithms that real spacecraft use to determine attitude and position from that sky: star-tracker simulation, lost-in-space attitude solvers, optical landmark / planet-limb navigation, and a closed-loop "navigate by stars" demo that corrects a deliberately noisy propagator.

These pair naturally: the star field is the rendering substrate, the astro-nav layer is the application that uses it for a meaningful purpose.

Pillar A — Star catalogue & renderer

5.1 Catalogue ingest

Pull the Hipparcos catalogue (HIP, ∼118 000 stars, magnitude limit ∼12.5) and the Yale Bright Star Catalogue (BSC5, ∼9 100 stars, magnitude limit ∼6.5 — the naked-eye sky).

- Download once via a manifest-driven script (mirroring `fetch_kernels.py`); store the parsed binary in `data/stars/` as packed Float32 columns (α, δ, V, $B-V$, ID)
- Sha256-pin in `backend/oamp/stars/manifest.toml`
- Re-export as a compact bundle for the browser (`web/public/data/stars_bsc5.bin`, ∼200 KB)

5.2 WebGPU star renderer

- Point-sprite pipeline: each star is a single instanced quad coloured by $B-V$ via a Planckian colour-temperature lookup, sized by magnitude
- Stars sit at the camera-clip far-plane in the inertial frame, so they don't move with camera position (only orientation)
- Optional constellation outlines (88 IAU constellations) toggled via a footer pill
- Optional star labels for Bayer designations (greek letters) at the ∼50 brightest — togglable; uses canvas-rendered text quads
- Magnitude-limit slider in the footer ($V_{\text{lim}} \in [-2, 7]$) lets the user dim the field for readability

5.3 Bright movers

Sun, Moon, planets are not in the star catalogue but should appear in the same view. Pull positions from SPICE at the active trajectory's time, render as small coloured discs sized by apparent magnitude, label them, and slide their positions as the time scrubber moves — so the user sees the Moon crossing a constellation across a multi-day trajectory.

Pillar B — Astro-navigation algorithms

5.4 Star-tracker simulation

Given a body-frame attitude quaternion $q_{\mathrm{B/I}}$ and a tracker FOV θ, return the list of catalogue stars in the FOV with simulated pixel coordinates and a noise model ($\sigma_{\mathrm{centroid}} \sim 0.5\,\mathrm{arcsec}$, optional bias). Endpoint `POST /astro/tracker`:

```
{
  "q_bi":  [w, x, y, z],
  "fov_rad": 0.349,                # 20 deg
  "mag_limit": 6.0,
  "noise_arcsec": 0.5
} -> [{"hip_id", "ra", "dec", "mag", "pixel_x", "pixel_y"}]
```

5.5 Lost-in-space attitude solver

Given a list of measured star pairs (pixel positions with no prior attitude knowledge), determine attitude using one of two classic algorithms:

- **Triangle / pyramid**: compute pairwise inter-star angles, look up against a K-vector indexed catalogue, vote for the most-consistent attitude
- **Davenport q-method / QUEST**: from three or more identified stars, solve the Wahba problem for the best-fit attitude quaternion

Endpoint `POST /astro/identify` returns the matched stars and the estimated attitude with a covariance.

5.6 Optical landmark / planet-limb navigation

When the spacecraft has prior attitude knowledge but unknown position:

- Render a planet from a known position using its SPICE-derived shape model; extract limb pixels
- Given measured limb pixels, recover line-of-sight to the planet centre, combine with parallax-against-stars and a coarse ephemeris prior, solve for spacecraft position
- This is what *Deep Space 1*, *Stardust*, and JAXA's Hayabusa missions all used ("AutoNav"-style optical navigation)

5.7 Closed-loop "navigate by stars" demo

Tie the pieces together:

1. Run a Phase 2 propagation as ground truth
2. Add Gaussian process noise to the propagator state to simulate unmodelled dynamics
3. At each "measurement epoch", simulate the star tracker (5.4) and an optical-navigation fix (5.6)
4. Run an extended Kalman filter (EKF) that fuses the noisy propagator with the measurements to estimate state + covariance
5. Render the truth orbit, the propagator estimate, and the EKF-corrected estimate as three overlaid trajectories with a 3σ uncertainty tube

The point: showcase a real autonomous-navigation loop ending in a sub-kilometre position covariance even when the propagator has been deliberately broken — the standard cislunar / deep-space navigation demonstration.

Pillar C — Frontend integration

5.8 Sky toggle + sandbox

- Header toggle: *Sky* on/off (default on once Phase 5 ships)
- Footer: magnitude-limit slider, constellation toggle, label toggle
- "Star-tracker view" camera mode that snaps the camera to the spacecraft's body-frame $+Z$ axis with the configured FOV — gives the user a literal look-through-the-tracker view, with the simulated noise overlay rendered as little crosshairs on each matched star
- Navigation-demo panel: choose a baseline trajectory, set noise / measurement rate, hit *Run* and watch the 3-trajectory comparison stream in

Sequencing

Work item	Est. duration	Dependency
5.1 Catalogue ingest	2–3 days	—
5.2 WebGPU star renderer	4–5 days	5.1
5.3 Bright movers (SPICE)	1–2 days	5.2 + existing SPICE
5.4 Star-tracker sim	2–3 days	5.1
5.5 Attitude solver	4–5 days	5.4
5.6 Limb navigation	4–5 days	5.2, 5.4
5.7 Nav demo + EKF	4–5 days	5.4, 5.6
5.8 Frontend integration	3–4 days	all
Total	**4–5 weeks**	

Hard tradeoffs decided up front

- **Catalogue choice**: BSC5 (Yale Bright Star, ∼9 K stars) for the rendered field — big enough to look like the real sky, small enough to ship in the public folder. HIP for the tracker simulation if a user wants tighter FOVs at fainter magnitudes.
- **Constellation outlines**: render from the standard IAU 88-constellation line list (Roger Sinnott's compilation); ship as a hand-curated JSON.
- **Time-dependence**: stars are treated as fixed at the J2000 equator/equinox. Proper motion is in the catalogue but ignored for rendering; star-tracker simulation may apply it for missions spanning years.
- **No barycentric aberration** for the renderer (negligible at spacecraft velocities). The tracker simulation *will* apply it when needed for sub-arcsecond residuals.
- **Attitude representation**: quaternions on the wire (always); DCM and Euler angles only inside specific algorithms.

Future / Stretch Goals

These are not scheduled. They are deliberately ambitious targets that would take OAMP from a Newtonian-with-CR3BP sandbox into territory currently inhabited by JPL Monte, GMAT, FreeFlyer, and AGI/STK Premium.

Stretch A — General Relativity

The original Phase 5 plan, demoted to a stretch goal in favour of the star-field / astro-nav phase. The work-breakdown below is preserved for when an interested contributor takes it on.

A.1 Post-Newtonian corrections (bridge)

Additive accelerations on top of the existing DOP853 integrator, validated against analytic results in the $c \to \infty$ limit and the canonical solar-system PN benchmarks.

- **Schwarzschild (1PN)**: dominant solar-system GR effect, $\mathbf{a}_{\text{Schw}} = \frac{\mu}{c^2 r^3}\left[(4\frac{\mu}{r} - v^2)\mathbf{r} + 4(\mathbf{r}\cdot\mathbf{v})\mathbf{v}\right]$
- **Lense–Thirring** frame-dragging from the central body's angular momentum $\mathbf{S} = I_\oplus \boldsymbol{\omega}_\oplus$
- **de Sitter** geodetic precession from the solar barycentric motion of the Earth–spacecraft system
- **Tests**: Mercury perihelion $42.98''$/century to $\pm 0.5\%$; Gravity Probe B geodetic 6606.1 mas/yr to $\pm 1\%$; Shapiro delay for Earth–Moon baseline; PN $\to 0$ as $c \to \infty$

A.2 Full GR geodesic integrator

DSX / Brumberg–Kopejkin BCRS metric: $ds^2 = -(1-2w/c^2)c^2dt^2 + (1+2w/c^2)\delta_{ij}dx^i dx^j + \mathcal{O}(c^{-4})$ with $w = \sum_A GM_A/r_A$ over solar-system bodies.

- 8D geodesic state $(t_{\text{TDB}}, x^\mu, u^\mu)$ integrated by DOP853, with $g_{\mu\nu}u^\mu u^\nu + c^2$ logged as a quality invariant
- Christoffel symbols computed analytically from the PN metric expansion — no numerical differentiation
- Endpoint `POST /propagate/gr` accepts BCRS initial conditions and a coordinate-time span

A.3 Tetrad thrust model

Convert body-frame thrust commands to the local orthonormal tetrad $e^\mu{}_{(a)}$ built from u^μ and the attitude quaternion; inject as the right-hand side of the non-geodesic equation $Du^\alpha/d\tau = f^\alpha$.

A.4 Metric pre-bake cache (Ray)

Live SPICE-driven $w(\mathbf{r}, t)$ evaluation is $\sim$100$\times$ slower than Newtonian propagation. A Ray actor pool samples w and its spatial gradients on a Cartesian grid over the mission window, stored as HDF5 in `data/metric_cache/` with a SHA256 manifest pinning the SPICE kernels used at bake time.

A.5 Frontend

GR vs PN vs Newton three-way overlay, residual panel showing $|\mathbf{r}_{\text{GR}} - \mathbf{r}_{\text{Newton}}|(t)$, proper-time τ readout alongside coordinate time, optional 2D equatorial slice of the Newtonian potential as a heatmap underlay.

Stretch B — Tesseral gravity

Beyond the zonal J_n, the C_{nm}, S_{nm} tesseral harmonics matter for GEO station-keeping (the 75°E and 105°W gravitational wells), Earth–Moon mascon flybys, and any sub-kilometre precision orbit determination on a non-spherical body. EGM2008 / GRGM1200A coefficients are public.

Stretch C — Ground-track & visibility analysis

- Project the trajectory onto an equirectangular Earth map; mark ascending / descending nodes
- Ground-station catalogue + line-of-sight pass prediction with elevation / azimuth windows
- Eclipse map: Earth and Moon shadow cones intersected with the trajectory, with umbra / penumbra distinction
- Sun-angle windows for solar-array sizing

Stretch D — Conjunction analysis

Against a TLE catalogue (Celestrak active satellites + debris): closest- approach distances over a 7-day prediction window, probability of collision under a 1σ covariance model, sortable risk list. This is the core service Combined Space Operations Centre (CSpOC) sells to operators.

Stretch E — Multi-spacecraft & constellation propagation

Propagate N spacecraft jointly, render coverage maps over a target region, plot constellation health (gap analysis, mean revisit time). Walker / Flower / Star-of-stars constellation generators as preset buttons.

Let Your Creativity Go Wild

These are not roadmap items. They are deliberately open-ended ideas where the only design constraint is "it should be cool." Pick one, build it, open a PR. Add new ideas as they occur to you.

Cinematic & storytelling modes

- **Tour mode** — auto-fly the camera through canonical historical missions (Apollo 11 TLI; Voyager 2 Jupiter encounter; Hayabusa 2 touchdown; JWST L_2 insertion; Hiten ballistic capture) with synchronised captions and timeline annotations
- **Procedural mission cinematography** — render a 60-second MP4 from any user trajectory with camera moves, captions, and a music bed selected from BPM-matched royalty-free libraries
- **Mission narrative export** — pull a 2-paragraph trip summary from the propagator data: "departed LEO 2026-01-04 17:30Z, performed a 3.1 km/s TLI burn, encountered the lunar SOI 4.2 days later..."

Live data & real-world coupling

- **Live-ISS mode** — pull the current ISS TLE every few hours; render its present position with a sun-illumination indicator and the orbital ground track of the next six passes
- **AR through your phone** — point the camera at the night sky; overlay predicted satellite paths and the OAMP-rendered spacecraft on the live video feed
- **Live debris catalogue** — render the ∼25 000 TLE-tracked objects as a particle cloud; click any one for identification, mass, country of origin
- **Solar-weather coupling** — pull NOAA F10.7 and Kp live; show their effect on MSIS drag in a side panel

Wild physics / alternate sandboxes

- **Alternate-physics sandbox** — "what if G was 2×?"; "what if the Moon weren't there?"; "what if Mercury orbited with PN turned off?" Change the constants, re-propagate, watch the orbit drift
- **Lyapunov-spectrum visualiser** — colour each piece of synodic-frame state space by its local Lyapunov exponent; chaotic regions around L_3 become visually obvious
- **N-body chaos sandbox** — drop N point masses with arbitrary positions / velocities, integrate, render the chaotic evolution and a Poincaré section
- **Light-sail / electric-propulsion** — low-thrust spiralling with shape-based methods; tightly coupled to a power model that follows SRP shadow and orientation
- **Interstellar trajectory mode** — Voyager-style outbound with PN-corrected acceleration tables; render the trajectory through the heliopause

New audiences & interfaces

- **WebXR mode** — view the scene in a VR headset; reach into the orbit and drag the spacecraft node to retime a maneuver

- **Audio synthesis** — drive a synth from orbital frequencies, eccentricity modulation, eclipses-as-amplitude. Kepler's "music of the spheres" but literal
- **Deep-field renderer** — JWST-style starfield with diffraction spikes, rendered when the camera looks away from any solar-system body. Optionally include nebulae (Hubble HST composites) as static panoramas
- **Multiplayer mode** — multiple users edit the same mission plan in real time (CRDT-backed editor state); chat sidebar with annotations pinned to orbit segments
- **Speedrun mode** — "land a payload in low lunar orbit in the minimum number of clicks"; leaderboard of optimised mission plans by total Δv, by time, by number of burns

Asteroid & deep-space

- **Asteroid-mining sandbox** — pull JPL Horizons NEAs, plan rendezvous, return-trajectory Δv budgets, payload mass fractions, accumulated regolith mass returned per dollar of fuel
- **Defending Earth** — given an inbound NEA on a hazardous approach, design a kinetic-impactor mission within a 12-month window; visualise the deflection cone of resulting orbits
- **Hayabusa-replica autonav** — close the loop end-to-end: propagate truth, simulate the optical-nav fix on the asteroid limb, fuse with the EKF (Phase 5), execute the touch-and-go

In-browser optimisation playground

- Drag a target marker around the 3D scene; watch the multi-burn NLP re-solve in real time with a colour gradient on the trajectory encoding $|\Delta v|$ density
- Pareto-front explorer — plot Δv vs. time-of-flight for a swept Lambert / patched-conic transfer; click any point to load the corresponding trajectory
- Sensitivity heatmap — perturb each design variable by 1σ; render the resulting trajectory bundle as a translucent tube

The point of this section is to keep the imagination ahead of the schedule. Every shipped roadmap phase started in a list like this.

Overall Timeline

Phase comparison

	P1	P2	P3	P4	P5
Physics	Newton + J_2	Newton + $J_2 \dots J_6$ + drag + SRP + 3B	= P2 (no new physics)	CR3BP + BCR4BP	= P4 (no new physics)
Frames	J2000	J2000	+ body selector (Earth/Moon)	+ EM synodic	+ body-frame attitude (quaternions)
Integrator	DOP853	+ Verlet + Yoshida-4	= P2	= P2 (CR3BP non-dim)	= P4
Optimisers	—	Lambert + Hohmann + CasADi NLP	all four exposed in sidebar; + TLE ingest	+ diff. correction (Lyapunov)	+ Wahba / QUEST attitude solver; EKF
Frontend	3 demo scenes	Maneuver editor + scrubber	Solver panel, Moon render, cislunar demo, apse markers, altitude chart, camera targeting	Frame selector, L_n markers, Lyapunov + manifold + WSB panels, animated play/pause, past-solid/future-dashed split, coastline overlay	Real celestial sphere, constellations, planet movers, star-tracker view, nav-demo 3-trajectory overlay
Heavy compute	—	Ray streaming	—	—	—
Status	*shipped*	*shipped*	*shipped*	*shipped*	next, ∼4–5 weeks

Critical path

```
P1  P2  P3  P4  P5 (star field + astro-nav)

                                    Stretch A: PN + GR
                                        Stretch B: tesseral gravity
                                        Stretch C: ground tracks
                                        Stretch D: conjunction analysis
                                        Stretch E: constellations
```

Phase 5 is the immediate next deliverable. The Stretch A–E follow-ups are independent of each other and can be picked up by different contributors once Phase 5 ships. GR (Stretch A) is the

most physically ambitious; the others are scoped tightly enough to be one-contributor weekend projects.

Conventions Reference

Convention	Value
Licence	AGPL-3.0 (network-service source-disclosure obligation; switched from Apache-2.0 during Phase 4 polish)
Time on wire	UTC ISO-8601 string
Time internal	TDB seconds since J2000 (= SPICE ET); see `oamp.timescales`
Units	SI throughout the engine (m, m/s, kg, s)
Accelerator	JAX (pixi `gpu` feature); CPU default
Heavy compute	Ray (pixi `full` feature)
SPICE kernels	Pinned by SHA256 in `backend/oamp/spice/manifest.toml`; *tracked in-tree* since the data-bundling pass so fresh clones run without network fetches
Coastline data	Natural Earth 110m, tracked at `web/public/data/`
Epoch J2000	2000-01-01T11:58:55.816Z UTC
$TT - TAI$	32.184 s (fixed by IAU)